

- Sears Home Services - Comprehensive Project Analysis
 - Executive Summary
 - 1. Technology Stack
 - Core Framework
 - Styling & UI
 - Data Management
 - Web Scraping & Data Collection
 - Additional Libraries
 - 2. Project Architecture
 - 2.1 Directory Structure
 - 2.2 Architectural Patterns
 - A. File-Based CMS Architecture
 - B. Dynamic Routing Strategy
 - C. Data Parsing Layer
 - 3. Core Functionality
 - 3.1 Symptom Center (/symptom-center)
 - 3.2 Repair Services (/repair)
 - 3.3 Appointment Scheduling (/scheduler)
 - 3.4 Order Management (/order)
 - 3.5 Maintenance Services (/maintain)
 - 3.6 Location Pages (/locations)
 - 3.7 Additional Features
 - 4. Data Management Approach
 - 4.1 Master Data Index
 - 4.2 Symptom Data Structure
 - 4.3 Scraped Data Structure
 - 4.4 Appointment Storage
 - 5. Workflow & User Journeys
 - 5.1 Symptom Diagnosis Journey
 - 5.2 Repair Service Discovery
 - 5.3 Appointment Booking Journey
 - 5.4 Order Management Journey
 - 6. Key Approaches & Patterns
 - 6.1 Data-Driven Page Generation
 - 6.2 Type-Safe Data Layer
 - 6.3 Server Components & Server Actions
 - 6.4 Component Composition

- 6.5 Progressive Enhancement
- 6.6 Scraping & Data Pipeline
- 7. Technical Highlights
 - 7.1 Dynamic Metadata Generation
 - 7.2 Smart File Resolution
 - 7.3 Attribute Parsing
 - 7.4 Node Filtering & Ordering
- 8. Development Workflow
 - 8.1 Adding New Content
 - 8.2 Data Updates
 - 8.3 Scripts Available
- 9. Strengths & Architecture Decisions
 -  Strengths
 -  Key Architecture Decisions
- 10. Potential Improvements & Considerations
 - Areas for Enhancement
- 11. Conclusion

Sears Home Services - Comprehensive Project Analysis

Note for Office Viewers: This document is in Markdown format (.md). Microsoft Word and other Office applications can open this file directly. If you see markdown syntax (like ****bold**** or **# headings**), you can use Word's "Open" feature which will attempt to render the markdown, or view it in a markdown viewer for better formatting.

Executive Summary

Sears Home Services is a Next.js-based web application designed for home appliance repair services, maintenance scheduling, and diagnostic information. The platform uses a **file-based data architecture** with dynamic routing to generate thousands of SEO-optimized pages from structured JSON data.

1. Technology Stack

Core Framework

- **Next.js 16.0.8** (App Router) - Server-side rendering and static generation
- **React 19.2.1** - UI library
- **TypeScript 5** - Type-safe development

Styling & UI

- **Tailwind CSS 4.1.17** - Utility-first CSS framework
- **Shadcn UI** (New York style) - Component library built on Radix UI
- **Radix UI** - Accessible component primitives (Accordion, Dialog, Dropdown, Select, Tabs, Tooltip)
- **Framer Motion 12.23.25** - Animation library
- **Lucide React** - Icon library

Data Management

- **File-based JSON storage** - No traditional database
- **Node.js filesystem APIs** - For reading/writing JSON files
- **Custom parsers** - Transform scraped HTML-like JSON into React components

Web Scraping & Data Collection

- **Puppeteer 24.34.0** - Browser automation for scraping
- **Playwright 1.57.0** - Alternative scraping tool
- **jsdom 27.4.0** - DOM parsing

Additional Libraries

- **react-markdown** - Markdown rendering
- **recharts** - Data visualization
- **embla-carousel-react** - Carousel components

- **react-scroll** - Smooth scrolling

2. Project Architecture

2.1 Directory Structure

```
searchhomeservices/
├── app/                                # Next.js App Router pages
│   ├── about/                         # About page
│   ├── blog/                          # Blog section
│   ├── contact/                       # Contact page
│   ├── glossary/                      # Glossary terms
│   ├── help/                          # Help center
│   ├── home-warrenty/                 # Home warranty pages
│   ├── locations/                     # Location-based pages
│   ├── maintain/                      # Maintenance services
│   ├── order/                         # Order lookup/management
│   ├── repair/                        # Repair service pages
│   ├── scheduler/                     # Appointment booking
│   ├── symptom-center/                # Symptom diagnosis center
│   ├── components/                   # Shared app components
│   ├── services/                     # Server actions/services
│   └── utils/                         # Utility functions
├── components/                        # Global reusable components
│   ├── ui/                           # Shadcn UI components
│   ├── Navbar.tsx
│   ├── Footer.tsx
│   └── ...
├── data/                              # JSON data files (file-based CMS)
│   ├── brands/                       # Brand-specific symptom data
│   ├── scraped/                       # Scraped repair service data
│   ├── maintain/                      # Maintenance service data
│   ├── brands_appliances_symptoms_structured.json # Master index
│   ├── appliance_issues_full.json
│   └── appointments.json              # Appointment storage
├── lib/                               # Shared utilities
├── utils/                             # TypeScript utilities & parsers
├── scripts/                           # Data scraping & processing scripts
└── public/                            # Static assets
```

2.2 Architectural Patterns

A. File-Based CMS Architecture

- **No Database:** Uses JSON files as the data source
- **Benefits:**
 - Fast read times
 - Version-controllable content
 - Easy integration with scraping scripts
 - No database migrations needed

B. Dynamic Routing Strategy

The app uses Next.js dynamic routes to handle thousands of pages:

- `/symptom-center/[brand-appliance-issue]` - Dynamic symptom pages
- `/repair/[repairService]/[appliance-repair-service]` - Brand-specific repair pages
- `/maintain/[slug]` - Maintenance service pages
- `/locations/[state]/[city]` - Location-based pages

C. Data Parsing Layer

Sophisticated parsers transform raw JSON (HTML-like node structures) into typed React components:

- `symptom-data.ts` - Parses symptom page data
- `repair-service-parser.ts` - Parses repair service pages
- `brand-appliance-parser.ts` - Parses brand-appliance specific pages

3. Core Functionality

3.1 Symptom Center (`/symptom-center`)

Purpose: Help users diagnose appliance issues by brand and appliance type.

Workflow:

1. User selects a **brand** from dropdown
2. User selects an **appliance** (filtered by selected brand)
3. System displays available **symptoms/issues** for that combination
4. User clicks on a symptom to view detailed diagnostic page

Key Components:

- `SymptomSection.tsx` - Brand/appliance selection interface
- `DynamicSymptomRenderer.tsx` - Renders symptom detail pages
- `getSymptomData()` - Fetches and parses symptom JSON files

Data Flow:

```
User Selection → brandAppliances.ts → Filter Symptoms →  
Load JSON from data/brands/[brand]/[appliance]/symptoms/[issue].json →  
Parse with symptom-data.ts → Render with DynamicSymptomRenderer
```

URL Structure: `/symptom-center/[brand]-[appliance]-[issue]` Example:
`/symptom-center/kenmore-washer-not-spinning`

3.2 Repair Services (`/repair`)

Purpose: Display repair service information for specific brand-appliance combinations.

Workflow:

1. User navigates to `/repair/[brand]/[appliance]`
2. System loads scraped data from
`data/scraped/[brand]/appliances/[appliance].json`
3. Parser extracts sections (hero, FAQ, resources, etc.)
4. Components render structured content

Key Components:

- `BrandApplianceRepairService.tsx` - Main renderer
- `parseRepairServiceData()` - Data parser
- `ImageScheduleCard` - CTA component

Data Structure:

- Scraped HTML-like JSON stored in `data/scraped/`
- Contains structured nodes with tags, content, attributes
- Parser extracts: hero data, FAQs, resources, brand suggestions, etc.

3.3 Appointment Scheduling (`/scheduler`)

Purpose: Multi-step appointment booking flow.

Workflow (5 steps + success):

1. **Step 1 - Product Selection:** Select brand and appliance
2. **Step 2 - Zip Code:** Enter service location
3. **Step 3 - Date/Time:** Select appointment slot
4. **Step 4 - Service Call Details:** Additional information
5. **Step 5 - Personal Information:** Contact details and address
6. **Step 6 - Success:** Confirmation screen

Key Features:

- Progress bar with step validation
- Summary sidebar showing selected options
- Form validation at each step
- Server action for appointment creation

Data Storage:

- Appointments stored in `data/appointments.json`
- Server actions in `app/services/appointmentService.ts`
- CRUD operations: create, read, update, delete, search

Key Components:

- `SchedulerPage.tsx` - Main container
- `DynamicFormContainer.tsx` - Step renderer
- `StepProduct.tsx`, `StepZipCode.tsx`, `StepDateTime.tsx`, etc.
- `SummarySidebar.tsx` - Booking summary
- `AntiGravityProgressBar.tsx` - Progress indicator

3.4 Order Management (`/order`)

Purpose: Lookup and manage existing appointments/orders.

Features:

- Order lookup by phone/email
- View appointment details
- Reschedule appointments

- Cancel appointments with reason tracking
- Upload appliance images (model number, serial, issue, barcode)

Key Components:

- `OrderLookup.tsx` - Search interface
- `OrderResultList.tsx` - Search results
- `OrderDetails.tsx` - Appointment details view
- `ActionFooter.tsx` - Reschedule/cancel actions

3.5 Maintenance Services (`/maintain`)

Purpose: Display maintenance service information and booking.

Structure:

- Dynamic pages from `data/maintain/` JSON files
- Adapters transform data into consistent format
- Components: Hero, Feature lists, Service cards, Booking cards

3.6 Location Pages (`/locations`)

Purpose: Location-specific service pages for SEO and local targeting.

Structure:

- `/locations/[state]` - State-level pages
- `/locations/[state]/[city]` - City-level pages
- City-specific content and CTAs

3.7 Additional Features

- **Blog Section** (`/blog`) - Content marketing
 - **Glossary** (`/glossary`) - Term definitions
 - **Help Center** (`/help`) - FAQ and support
 - **Home Warranty** (`/home-warrenty`) - Warranty information
 - **About/Contact** - Company information
-

4. Data Management Approach

4.1 Master Data Index

File: `data/brands_appliances_symptoms_structured.json`

This is the **Single Source of Truth** for brand-appliance-symptom relationships:

```
[
  {
    "brand": "Kenmore",
    "appliances": [
      {
        "appliance": "Washer",
        "symptoms": ["not-spinning", "leaking", "not-draining"]
      }
    ]
  }
]
```

Usage:

- Powers all dropdown selections
- Ensures consistency across components
- Exported via `utils/brandAppliances.ts` with TypeScript types

4.2 Symptom Data Structure

Location: `data/brands/[brand]/[appliance]/symptoms/[issue].json`

Format: HTML-like node structure:

```
[
  {
    "full_content": [
      {
        "tag": "h1",
        "content": "Kenmore Washer Not Spinning",
        "attributes": {},
        "order": 1
      },
      {
        "tag": "h2",
```

```
        "content": "Common Causes",
        "attributes": {},
        "order": 2
    }
]
}
```

Parser: `app/utils/symptom-data.ts` extracts:

- Meta (title, description)
- Statistics
- Repair steps
- FAQs
- Testimonials
- Glossary terms
- Cross-links
- Blog posts
- Quick repair guides

4.3 Scraped Data Structure

Location: `data/scraped/[brand]/appliances/[appliance].json`

Format: Similar node structure to symptom data **Parser:** `utils/repair-service-parser.ts` extracts:

- Hero data (heading, description, image)
- Random content sections
- FAQ data
- Brand suggestions
- Common symptoms
- Repair resources
- Expert information

4.4 Appointment Storage

File: `data/appointments.json`

Structure:

```
interface Appointment {  
  id: string;  
  createdAt: string;  
  updatedAt: string;  
  appliance: string;  
  brand: string;  
  zipCode: string;  
  serviceDate: string;  
  serviceTime: string;  
  firstName: string;  
  lastName: string;  
  email: string;  
  phone: string;  
  streetAddress: string;  
  city: string;  
  state: string;  
  status: 'upcoming' | 'visited' | 'cancelled';  
  // ... additional fields  
}
```

Operations: Server actions handle CRUD operations

5. Workflow & User Journeys

5.1 Symptom Diagnosis Journey

1. Landing Page (HeroSection)
↓
2. Select Brand → Select Appliance
↓
3. Symptom Center Page (SymptomSection)
↓
4. View Available Symptoms
↓
5. Click Symptom → Dynamic Symptom Page
↓
6. Read Diagnostic Information
↓
7. Click "Schedule Now" → Scheduler Flow

5.2 Repair Service Discovery

1. Navigate to /repair/[brand]/[appliance]
↓
2. View Repair Service Page
 - Hero section with CTA
 - Service information
 - FAQs
 - Related resources↓
3. Click "Schedule Repair" → Scheduler Flow

5.3 Appointment Booking Journey

1. Enter Scheduler (/scheduler)
↓
2. Step 1: Select Brand & Appliance
↓
3. Step 2: Enter Zip Code
↓
4. Step 3: Select Date & Time
↓
5. Step 4: Service Call Details
↓
6. Step 5: Personal Information
↓
7. Submit → Appointment Created
↓
8. Success Screen with Order ID
↓
9. Redirect to /order/[orderId]

5.4 Order Management Journey

1. Navigate to /order
↓
2. Enter Phone/Email to Search
↓
3. View Matching Appointments
↓
4. Select Appointment
↓
5. View Details
↓
6. Options:
 - Reschedule (new date/time)

- Cancel (with reason)
- Upload Images

6. Key Approaches & Patterns

6.1 Data-Driven Page Generation

Approach: JSON files automatically generate pages without code changes.

Benefits:

- Scalability: Add thousands of pages by adding JSON files
- SEO: Each page is a real route with metadata
- Maintainability: Content updates don't require code changes
- Version Control: Content changes tracked in Git

6.2 Type-Safe Data Layer

Approach: TypeScript interfaces for all data structures.

Files:

- `utils/brandAppliances.ts` - Brand/appliance types
- `app/utils/symptom-data.ts` - Symptom page types
- `utils/repair-service-parser.ts` - Repair service types
- `app/services/appointmentService.ts` - Appointment types

Benefits: Compile-time error checking, IDE autocomplete, refactoring safety

6.3 Server Components & Server Actions

Approach: Next.js App Router with server components and actions.

Usage:

- Server components for data fetching (no client JS needed)
- Server actions for mutations (appointment CRUD)

- Client components only where interactivity needed

Benefits: Better performance, SEO, reduced client bundle size

6.4 Component Composition

Approach: Small, focused components composed into larger pages.

Examples:

- `DynamicSymptomRenderer` composes multiple section components
- `DynamicFormContainer` switches between step components
- Reusable UI components from Shadcn UI

6.5 Progressive Enhancement

Approach: Forms and interactions work without JavaScript where possible.

Implementation:

- Server-side form handling
- Progressive validation
- Graceful degradation

6.6 Scraping & Data Pipeline

Approach: Automated data collection and processing.

Scripts:

- `batch-scrape-brands.ts` - Scrape brand pages
- `batch-scrape-repair-services.ts` - Scrape repair pages
- `format-scraped-data.ts` - Clean and structure data
- `summarize-brands.ts` - Generate summaries

Workflow:

```
Scrape HTML → Parse to JSON → Store in data/ →  
Parser transforms → React components render
```

7. Technical Highlights

7.1 Dynamic Metadata Generation

Implementation: `generateMetadata()` functions in page components.

Example:

```
export async function generateMetadata({ params }: PageProps) {
  const data = await getSymptomData(slug);
  return {
    title: data.meta.title,
    description: data.meta.description
  };
}
```

Benefit: SEO-optimized metadata for each dynamic page

7.2 Smart File Resolution

Approach: Multiple fallback strategies for finding data files.

Example (from `getSymptomData`):

```
const strategies = [
  `${issue}.json`,
  `${appliance}-${issue}.json`,
  `${brand}-${appliance}-${issue}.json`
];
```

Benefit: Handles various naming conventions, reduces 404s

7.3 Attribute Parsing

Challenge: JSON attributes stored as strings need parsing.

Solution: `parseAttributes()` function handles:

- String to object conversion
- Python-style booleans (True/False → true/false)
- None → null conversion
- Quote normalization

7.4 Node Filtering & Ordering

Approach: Filter and sort nodes by order property, remove unwanted sections.

Implementation:

- Sort by `order` field
 - Filter nodes before/after specific H2 tags
 - Remove footer/unwanted content sections
-

8. Development Workflow

8.1 Adding New Content

For Symptoms:

1. Add entry to `brands_appliances_symptoms_structured.json`
2. Create JSON file in
`data/brands/[brand]/[appliance]/symptoms/[issue].json`
3. Deploy - page automatically available

For Repair Services:

1. Run scraping script: `npm run scrape:brands`
2. Or manually create JSON in
`data/scraped/[brand]/appliances/[appliance].json`
3. Deploy - page automatically available

8.2 Data Updates

Process:

1. Update JSON files in `data/` directory
2. Commit changes
3. Deploy - changes reflected immediately

No migrations needed - file-based system is flexible

8.3 Scripts Available

```
npm run dev           # Development server
npm run build          # Production build
npm run scrape:cooktop # Scrape cooktop pages
npm run scrape:brands  # Batch scrape brand pages
npm run summarize:brands # Generate brand summaries
npm run format:scraped # Format scraped data
```

9. Strengths & Architecture Decisions



Strengths

1. **Scalability:** Can handle thousands of pages without code changes
2. **SEO-Friendly:** Each page is a real route with proper metadata
3. **Type Safety:** Comprehensive TypeScript coverage
4. **Performance:** Server-side rendering, minimal client JS
5. **Maintainability:** Content separate from code
6. **Version Control:** All content tracked in Git
7. **Flexibility:** Easy to add new content types



Key Architecture Decisions

1. **File-Based Over Database:**
 - Chosen for simplicity, speed, and version control
 - Trade-off: No real-time updates, but acceptable for this use case

2. Dynamic Routing Over Static Pages:

- Enables thousands of pages without code duplication
- Trade-off: Slightly more complex routing logic

3. Parser-Based Rendering:

- Transforms raw JSON into components
- Trade-off: More complex than direct rendering, but enables flexible content structure

4. Server Actions Over API Routes:

- Simpler for mutations, better type safety
 - Trade-off: Less flexible than REST API, but sufficient for needs
-

10. Potential Improvements & Considerations

Areas for Enhancement

1. Caching Strategy:

- Implement caching for frequently accessed JSON files
- Consider ISR (Incremental Static Regeneration) for symptom pages

2. Error Handling:

- More robust error boundaries
- Better 404 handling for missing data files

3. Data Validation:

- Schema validation for JSON files (e.g., Zod)
- Validate data structure at build time

4. Search Functionality:

- Full-text search for symptoms and repair guides

- Consider Algolia or similar

5. Analytics Integration:

- Track user journeys through symptom diagnosis
- Monitor booking conversion rates

6. Image Optimization:

- Ensure all images use Next.js Image component
- Implement proper lazy loading

7. Accessibility:

- Audit and improve ARIA labels
- Keyboard navigation improvements

8. Testing:

- Unit tests for parsers
- Integration tests for booking flow
- E2E tests for critical user journeys

11. Conclusion

The Sears Home Services platform demonstrates a well-architected, scalable solution for a content-heavy service booking application. The file-based CMS approach, combined with dynamic routing and sophisticated parsing, enables the platform to handle thousands of pages efficiently while maintaining type safety and developer experience.

The architecture prioritizes:

- **Scalability** through data-driven page generation
- **SEO** through proper routing and metadata
- **Type Safety** through comprehensive TypeScript usage
- **Performance** through server-side rendering
- **Maintainability** through separation of content and code

This approach is particularly well-suited for content that changes infrequently and benefits from version control, making it an excellent choice for a service-oriented platform with extensive product documentation.